Alen Da Luz
26 Aprile 2026

Report Tecnico

Policy & Packet Capture: Configurazione Firewall Windows, ICMP e Analisi Forense HTTPS con INetSim e Wireshark

1. Scenario Operativo e Contesto del Laboratorio

Il laboratorio contrappone un sistema **Kali Linux** a un endpoint **Windows**. Il protocollo **ICMP (Internet Control Message Protocol)** è il primo punto di verifica della comunicazione tra i due nodi. Per impostazione predefinita, Windows adotta una postura di sicurezza restrittiva, **bloccando le richieste ICMP in entrata** per ridurre la superficie di attacco.

È fondamentale distinguere tra **Inbound Rules (Regole in entrata)** e **Outbound Rules (Regole in uscita)**. La configurazione **"Default Deny"** sulle connessioni in entrata richiede un intervento manuale per autorizzare traffico specifico senza compromettere l'integrità del sistema.

Componente	Ruolo	Sistema / Indirizzo
Kali Linux	Attaccante / Analista	Debian-based
Windows Endpoint	Target / Vittima	IP: 192.168.50.102
Windows Defender Firewall	Controllo traffico inbound	Servizio MpsSvc

2. Diagnosi Iniziale: Stato della Connettività ICMP

Prima di apportare modifiche alle **policy di sicurezza**, è fondamentale stabilire una linea di base che confermi la raggiungibilità dell'host target. La fase diagnostica permette di distinguere tra un blocco a **livello applicativo (firewall)** e potenziali problemi ai livelli inferiori della pila **ISO/OSI**.

Il test consiste nell'inviare un **ICMP Echo Request** dalla macchina Kali Linux verso l'indirizzo **192.168.50.102**. I risultati confermano che il sistema target opera in stato di **hardening attivo**: il **Windows Defender Firewall** segue una policy di **"Default Deny"** e le richieste ICMP vengono ignorate (**dropped**) in modo silenzioso.

> Per ripristinare la comunicazione senza disattivare le protezioni globali, è necessario implementare una regola esplicita che autorizzi esclusivamente il traffico **ICMP Echo Request** proveniente dalla sorgente specifica.

3. Attivazione e Gestione del Servizio Windows Firewall (MpsSvc)

Il **Windows Defender Firewall** è gestito dal servizio **MpsSvc**. Se non è attivo, il sistema non può applicare né controllare le regole di sicurezza. La gestione avviene tramite la console **services.msc**.

I parametri da verificare:

- **Identificazione**: servizio con Nome **MpsSvc**, Nome visualizzato **Windows Firewall**.
- **Tipo di avvio**: nel laboratorio impostato su **Manuale**.
- **Stato**: verificare che sia **"Avviato"** prima di procedere.

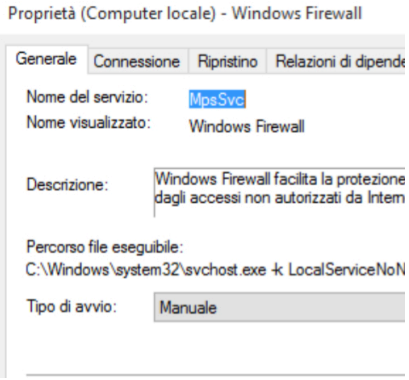

Fig. 1 — Console services.msc — proprietà del servizio MpsSvc: Windows Firewall, percorso svchost.exe, tipo di avvio Manuale

Una volta confermata l'attività del servizio **MpsSvc**, il sistema inizia a filtrare il traffico in base alle policy vigenti, rendendo necessaria la creazione di una regola specifica per il traffico ICMP.

4. Creazione della Regola Inbound "allow_ping"

L'obiettivo è creare una regola che permetta esclusivamente il comando ping, minimizzando i rischi di **movimento laterale**. La procedura si esegue tramite la **Creazione guidata nuova regola connessioni in entrata**.

Step-by-Step Configuration

Passaggio	Impostazione	Note operative
Tipo di regola	Personalizzata	Massimo controllo sui parametri
Programma	Tutti i programmi	Opera indipendentemente dall'eseguibile
Protocollo e Porte	ICMPv4 (Protocollo 1)	Filtra esclusivamente il traffico ping
Ambito — Lab	Qualsiasi indirizzo IP	Sorgente/destinazione libere in test
Ambito — Prod.	IP specifici / subnet	Limitare agli IP degli amministratori
Operazione	Consenti la connessione	Include connessioni protette e non
Profilo	Dominio, Privato, Pubblico	Continuità in tutti i contesti di rete
Nome	allow_ping	Identificativo univoco

Fig. 2 — Wizard — step 'Tipo di Regola': Personalizzata; step 'Programma': Tutti i programmi

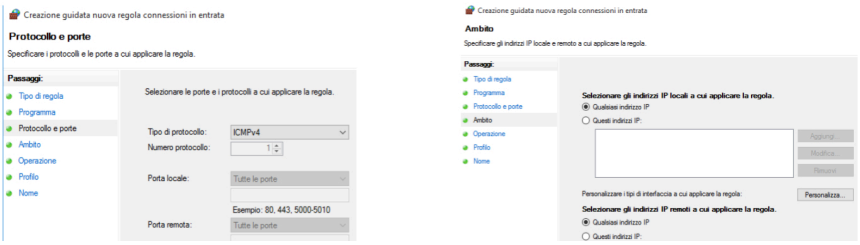

Fig. 3 — Wizard — step 'Protocollo e Porte': ICMPv4, protocollo 1; step 'Ambito': Qualsiasi indirizzo IP

Fig. 4 — Wizard — step 'Operazione': Consenti la connessione; step 'Profilo': Dominio, Privato e Pubblico

> **Security Best Practice**: In produzione l'**Ambito** remoto deve essere limitato alla subnet di gestione per prevenire ricognizione da host non autorizzati.

5. Verifica Post-Configurazione e Validazione

Al termine della procedura, la regola **allow_ping** appare nell'elenco **Regole connessioni in entrata**. Il pannello **Azioni** offre:

- **Disabilita regola**: sospende il permesso senza eliminare la configurazione.
- **Proprietà**: consente modifiche successive ai parametri.
- **Elimina / Taglia / Copia**: strumenti per la manutenzione ordinaria.

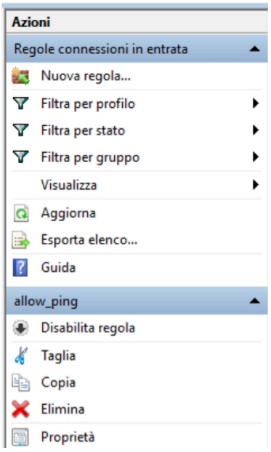

Fig. 5 — Windows Defender Firewall — allow_ping nell'elenco Inbound Rules; pannello Azioni con Disabilita, Proprietà, Elimina

Validazione Tecnica Finale

Il test eseguito da Kali Linux conferma il successo della configurazione. I tempi di risposta sono stabili: il firewall non introduce latenze significative.

```
$ ping -c 4 192.168.50.102
64 bytes from 192.168.50.102: icmp_seq=1 ttl=128 time=0.476 ms
64 bytes from 192.168.50.102: icmp_seq=2 ttl=128 time=1.54 ms
64 bytes from 192.168.50.102: icmp_seq=3 ttl=128 time=1.79 ms
64 bytes from 192.168.50.102: icmp_seq=4 ttl=128 time=1.29 ms
4 packets transmitted, 4 received, 0% packet loss, time 3021ms
rtt min/avg/max/mdev = 0.476/1.271/1.788/0.492 ms
```

0% packet loss e latenza media di **1.271 ms**: la regola allow_ping è operativa senza overhead significativo.

6. Simulazione Forense con INetSim: Obiettivi e Architettura

L'emulazione dei servizi di rete in un ambiente controllato è una metodologia critica per mitigare i **rischi operativi** nell'analisi malware. L'obiettivo specifico è isolare il traffico **HTTPS** per intercettare e analizzare le **transazioni cifrate**, garantendo l'integrità dei dati catturati per finalità forensi. L'architettura si compone di tre elementi:

- **Kali Linux**: sistema host per l'hosting dei servizi e l'esecuzione degli strumenti di analisi.
- **INetSim**: stanza di simulazione in **"fake mode"**, risponde alle richieste del client senza esporre infrastrutture reali.
- **Wireshark**: validazione del protocollo a livello di bit per la verifica forense del traffico.

Componente	Ruolo	Modalità Operativa
Kali Linux	Sistema host / piattaforma analisi	OS principale (Oracle VirtualBox)
INetSim 1.3.2	Simulatore servizi di rete	Fake mode — listener su 127.0.0.1
Wireshark	Packet analyzer / network forensics	Cattura su interfaccia loopback:lo

7. Preparazione dell'Ambiente e Hardening di INetSim

La manipolazione delle **configurazioni dei demoni** in directory protette come **/etc/** necessita di **utenza root**. Il terminale di sinistra mostra l'accesso al file di configurazione tramite **sudo nano** e l'avvio di INetSim; quello di destra mostra la struttura del file di configurazione.

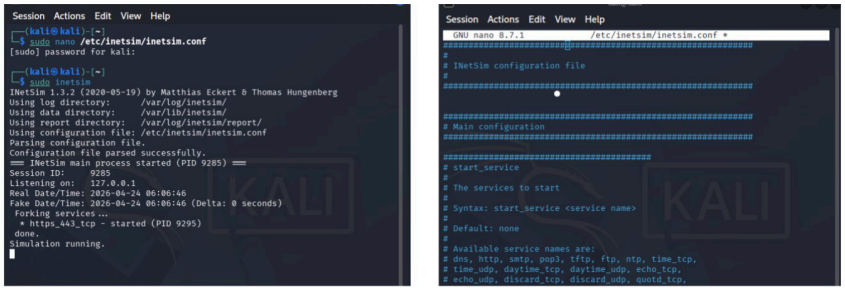

Fig. 6 — Terminale Kali — sudo nano /etc/inetsim/inetsim.conf e avvio INetSim (sx); file di configurazione inetsim.conf nel nano editor (dx)

L'analista deve operare una **riduzione dei protocolli attivi** applicando il **carattere #** per commentare tutti i servizi eccetto **HTTPS**. La riga **start_service https** rimane l'unico **listener attivo**:

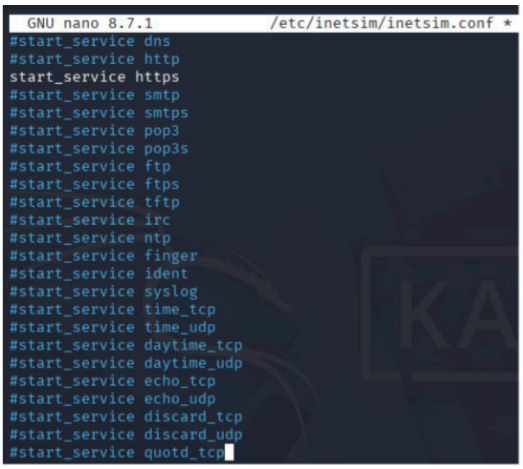

Fig. 7 — Nano editor — start_service https come unico listener attivo; tutti gli altri servizi commentati con #

Il servizio è inoltre vincolato all'indirizzo **127.0.0.1** (loopback), prevenendo qualsiasi **leak di pacchetti** verso reti esterne:

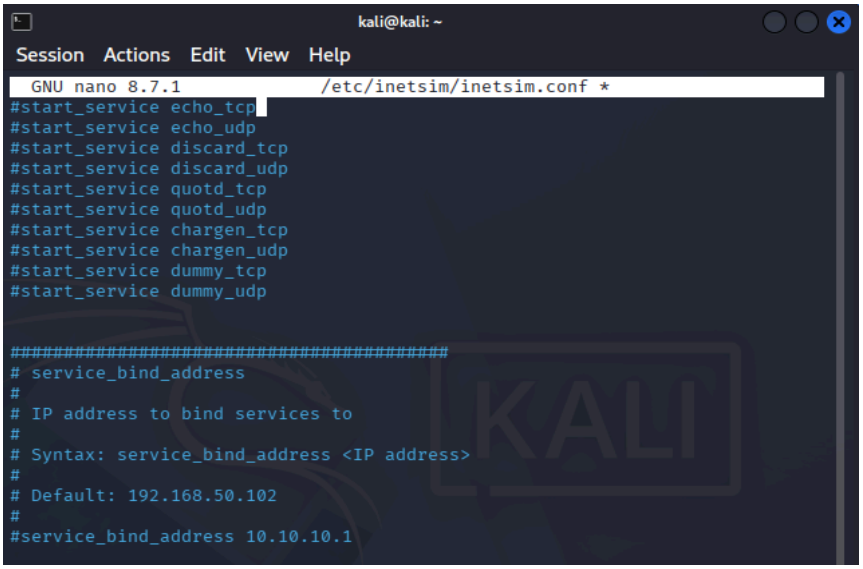

Fig. 8 — Nano editor — sezione service_bind_address: servizio configurato su 127.0.0.1 (loopback)

8. Deployment del Servizio e Analisi Forense con Wireshark

Salvate le modifiche, si procede all'attivazione del demone tramite **sudo inetsim**. Il binding avviene su **127.0.0.1** con istanziazione del processo **https_443_tcp**.

```
$ sudo inetsim
INetSim 1.3.2 — by Matthias Eckert & Thomas Hungenberg
Listening on:  127.0.0.1
=== INetSim main process started (PID 9285) ===
  * https_443_tcp — started (PID 9295)
done. Simulation running.
```

La verifica funzionale avviene accedendo a **https://127.0.0.1**. La risposta — pagina HTML predefinita del server INetSim in fake mode — impedisce al **software malevolo** di interrompere l'esecuzione per errori 404:

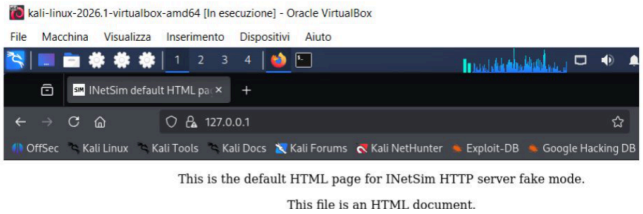

Fig. 9 — Browser Kali Linux su https://127.0.0.1 — pagina HTML predefinita INetSim fake mode

INetSim è in grado di servire **file "fake" con estensioni reali**, confermando la robustezza dell'ambiente nel gestire qualsiasi richiesta client simulata:

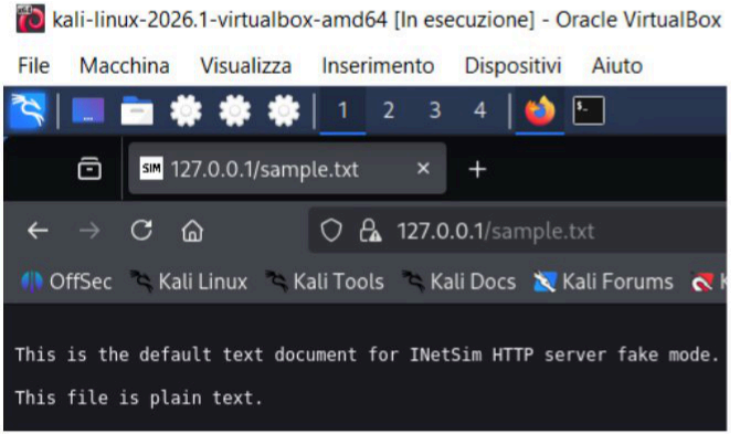

Fig. 10 — Browser Kali Linux su https://127.0.0.1/sample.txt — file fake con estensione reale

Analisi del Traffico e Network Forensics con Wireshark

La cattura sull'interfaccia **loopback:lo** verifica l'integrità della sessione. L'analisi del **TCP 3-Way Handshake** (SYN → SYN+ACK → ACK) seguita dai frame **TLSv1.3** conferma la corretta instaurazione del socket e la negoziazione cifrata:

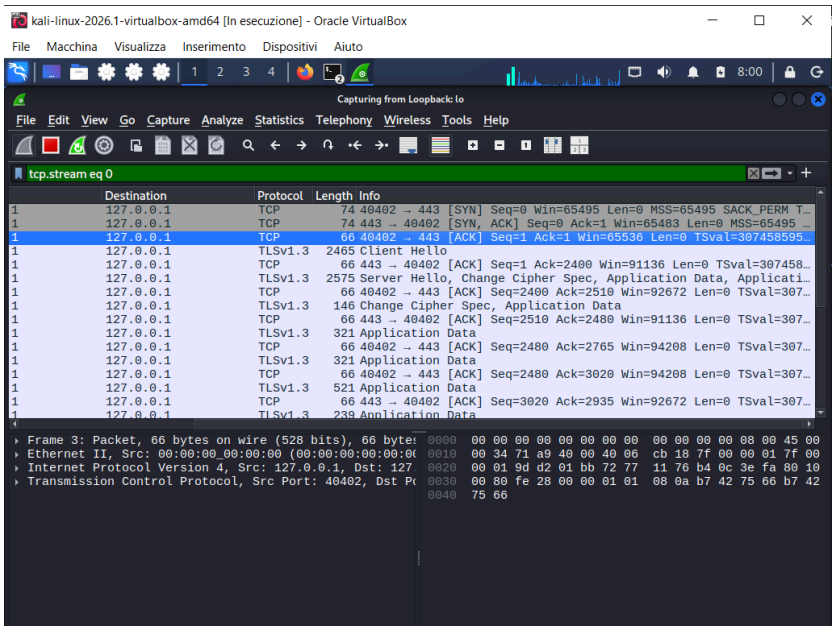

Fig.*11 — Wireshark — cattura loopback:lo: TCP 3-Way Handshake verso 127.0.0.1:443, frame TLSv1.3 Client Hello, Server Hello e Application Data*

Fase	Flag / Protocollo	Descrizione
1 — Apertura TCP	TCP SYN	Client richiede connessione a porta 443
2 — Risposta server	TCP SYN+ACK	INetSim accetta la connessione
3 — Handshake completato	TCP ACK	Canale TCP stabilito
4 — Negoziazione TLS	TLSv1.3 Client/Server Hello	Scambio parametri crittografici
5 — Sessione cifrata	TLSv1.3 Application Data	Dati HTTPS trasmessi cifrati

9. Conclusioni

Il progetto ha affrontato e risolto due distinte sfide operative, dimostrando l'importanza di un approccio sistematico alla configurazione della sicurezza di rete.

Nella **prima parte**, la comunicazione ICMP è stata stabilizzata tramite la regola **allow_ping** autorizzata dal servizio **MpsSvc**. L'intervento puntuale sulla policy di Default Deny ha ripristinato la raggiungibilità dell'host senza compromettere la postura di sicurezza complessiva.

Nella **seconda parte**, la configurazione di **INetSim** ha creato un ambiente di **analisi HTTPS forense** isolato. L'hardening selettivo con binding esclusivo su loopback e attivazione del solo listener HTTPS ha garantito la purezza della cattura. L'analisi Wireshark ha confermato la corretta instaurazione del socket TCP e la negoziazione **TLSv1.3**, validando la robustezza dell'ambiente nel gestire **richieste client simulate**.